

DeBait — System & Reliability Brief

Product engineering summary. Campaign figures are from local deterministic runs; the separate Stripe test-mode checkpoint and unmeasured paths are labeled explicitly.

The problem

Social-engineering attacks are not confined to one app. A single episode moves across a phone call, a chat message, a browser page, and a payment. The evidence that ties them together is lost at each hop, and defenders are left with weak signals — a matching amount, a coincident timestamp — that cannot safely justify an intervention. A matching amount is only a clue, not provenance.

One agent, a self-directed loop

DeBait is **one goal-directed agent** with a single objective: *contain the episode without disrupting legitimate activity*. It is not a fixed four-step pipeline. On every iteration it runs a self-directed loop and decides what to do next based on what it currently knows:

observe → reason → decide (read more vs. intervene) → policy gate → act → verify → adapt

- **Observe** — pull the next authorized piece of evidence it is missing, following only trusted cross-app links.
- **Reason** — interpret accumulated evidence; surface contradictions, missing evidence, and a bounded next read. Content is data and never grants authority.
- **Decide** — gather more evidence, intervene now, or escalate for review.
- **Policy gate** — authorize each action against trusted scope and exact bindings; injected or unlinked targets are refused.
- **Act** — contain the exact linked resources.
- **Verify** — re-read external state; an API acknowledgment is not a verified outcome.
- **Adapt** — reconcile uncertain/partial results and repeat until CONTAINED, PARTIALLY_CONTAINED, PREVENTION_FAILED, or REVIEW_REQUIRED.

Because the agent chooses its own next step, it diverges when the situation demands: if the attack is clear before a payment exists, it contains the live call/chat/browser and never touches Stripe; if the bound payment has already settled, it withholds a pointless cancel and escalates honestly rather than claiming a false success.

Four external apps — reads and actions

App	Agent observes	Agent can act
Twilio	call state / transcript evidence	end the specific bound call
Telegram	message / sender evidence	delete the designated message, ban the designated actor
Browserbase	managed session / page evidence	release (terminate) the exact managed session
Stripe (sandbox)	linked PaymentIntent and state	cancel the exact scam-linked test payment

The same incident produces coordinated, scoped writes across several systems — not merely a summary or alert.

Compact system path



Trust boundary: AI proposes, deterministic code authorizes

The agent (and its semantic reasoner) determine *what should happen next*. A deterministic policy and scoped broker determine *what is permitted*: every action is checked against the episode's trusted resource bindings and standing consent before dispatch. Payment targets are resolved only through stored trusted-origin edges, never from model prose. This is why attacker-controlled text cannot widen the agent's reach.

Inbound Stripe and Twilio callbacks verify provider signatures before resolving a trusted resource binding or persisting sanitized evidence; local tests cover tampering, stale signatures and ambiguous forms.

Proactive opt-in. A standing owner opt-in scopes DeBait to the owner's own resources and lets it intervene without a human "break the chain" button. It does not authorize arbitrary new targets or real-person outreach.

Persistence. Every event, edge, action, observation, and decision step is persisted in a durable episode store, and each episode exposes its full decision trace — the loop the agent actually followed is inspectable, not implied.

Architecture diagram: [architecture.html](#) (interactive; illustrative payloads, no API calls).

Evaluation method

Deterministic local fixture worlds drive the real policy, broker, worker, and independent-verification path from authored manifests. Each case declares its expected final state and world; a run is *correct* only when the episode state, the world, and the "no unrelated resource changed" check all match. Denominators are counted before execution. Repeat seeds are recorded, while current fixture execution remains deterministic.

Two manifests: a 5-case core baseline and a **48-case reliability campaign** — four pretext categories (bank impersonation, investment, tech support, fake emergency) × four families (scam, benign lookalike, integration fault, adversarial), 12 each — run at 3 repeats (144 executions).

Results — 48-case campaign, 144 executions (local fixture)

Metric	Result
Correct outcomes	144 / 144 (100%)
Recoverable attacks contained	84 / 84
Benign lookalikes uninterrupted	36 / 36
Integration-fault outcomes correct	36 / 36
False financial interventions	0
Unauthorized effects	0
Duplicate logical effects	0
Incorrect final states	0
Median / p95 containment latency	~0.20 s / ~0.37 s

The other 24 attack-family runs are integration faults designed to end partially contained (missing client proof) or prevention-failed (payment already settled). They are excluded from the recoverable denominator and included in correct-outcome accuracy, so the report does not hide best-possible non-contained outcomes.

Specific reliability properties

- **Two-payment targeting.** An equal-value unrelated payment is preserved in every run; only the trusted-edge-linked payment is cancelled.
- **Lost response & restart recovery.** A fault transport applies the cancellation then drops its success response; reconciliation re-reads authoritative state and records exactly **one** logical effect. Durable queue leases recover an in-flight action across a real process restart without duplicating it.
- **Verification levels.** Requested, acknowledged, read-back, and client-observed are distinct. Telegram deletion that is only acknowledged (no independent client observation) yields PARTIALLY_CONTAINED, not a false success.
- **Prompt-injection resistance.** Injected instructions (e.g. "cancel pi_unrelated", "reveal keys") never become a read or an action across all adversarial cases; unauthorized effects remain 0.
- **Budget-safe extraction reuse.** Validated model assessments are cached by canonical evidence, permitted reads, exact model and prompt version. Cache hits are revalidated, visibly labeled and add zero new tokens or cost; evidence or prompt changes force a new extraction.
- **Hunter isolation.** The decoy investigator runs only after victim-facing containment, in a separate context; it cannot reopen the victim session or move money.
- **Candidate gate (design).** Improvements are evaluated against a frozen held-out baseline; nothing auto-promotes.

Honest limitations

- **One live-provider checkpoint, not a live four-app loop.** A standalone Stripe test-mode run created two equal \$250 PaymentIntents, canceled and read back the exact scam-linked object, preserved the control, and correctly refused a repeated cancel. Telegram, Browserbase and Twilio remain mock-transport tested; the agent does not yet drive all four real adapters end to end.
- **No fresh (paid) model call is measured.** The budgeted model client is transport-tested offline; the same loop runs under it, but no live-key semantic evaluation has been performed.
- **Grok Bot adapter** is preserved as a provider-neutral target and remains access-dependent and unimplemented; a local adversary is not a completed Grok integration.
- The 48-case campaign is an authored deterministic fixture catalog, not a statistically sampled or independently reviewed dataset; confidence intervals are not reported.

Reproduce

- Backend: `uv run pytest -q` (199 tests) · `lint uv run ruff check .`
- Campaign: `uv run debait eval evals/manifests/full-campaign.json --repeats 3`
- Frontend: `npm --prefix frontend run build` and Playwright (12 tests)
- Every exported report records its source revision, fixture hash, seed, repeat count and limitations.